Project 1: From PCA to Neural Networks

Learning Objectives

By the end of this project, you should be able to:

1. Understand **PCA as a subspace method** and generalize it from vectors to **CNN filters**.
2. Implement **Decomposed Convolutional Filters (DCF)** and interpret them as principal components in filter space.
3. Analyze how **low-dimensional filter subspaces** affect reconstruction, classification, and domain generalization.
4. Apply filter decomposition ideas to **domain adaptation,** and **continual learning**.
5. Critically explore **alternative decompositions** beyond classical PCA
6. Apply the concepts and methods from this project to a research problem of your own interest.

Background & Reading:

- **DCFNet (ICML 2018)** [1] – filter decomposition as truncated basis expansion
- **Dictionary-based Domain-Invariant Learning (NeurIPS 2020)** [2] – domain-specific atoms + shared coefficients
- **Continual Learning with Filter Atom Swapping (ICLR 2022)** [3] – task-specific subspaces with shared coefficients
- **Image Generation using Continuous Filter Atoms (NeurIPS 2021)** [4] – generate filter atoms with a neural ODE
- **Reference GitHub Code**: https://github.com/ZeWang95/DCFNet-Pytorch, https://github.com/ZeWang95/ACDA

Part 1: PCA View of CNN Filters (DCF in AlexNet)

Goal: Show that **CNN filters live in a low-dimensional subspace**, and that truncating this subspace behaves like PCA.

Task 1: Principal Component Analysis of Filters

1. Extract trained convolutional filters: Train the AlexNet on MNIST dataset and save the trained AlexNet weights.

2. Perform PCA on convolutional filters. Replace the convolutional layers $W \in \mathbb{R}^{c' \times c \times l \times l}$ in AlexNet with a DCF layer: $W = \sum_{k=1}^{K} a_k \psi_k$, where $\psi_k \in \mathbb{R}^{l \times l}$ are filter atoms, $a_k \in \mathbb{R}^{c' \times c}$ are atom coefficients, c' and c are the size of input and out channel, $l \times l$ is the size of convolutional kernels. (Note: to get a_k and ψ_k, you should apply PCA to reshaped $W' \in \mathbb{R}^{c'c \times ll}$).

3. Vary the number of filter atoms (K) and make plots:
    - Test accuracy on MNIST vs number of filter atoms.
    - Number of parameters vs number of filter atoms.

Task 2: Compare Filter Atoms with Fourier-Bessel Bases

1. Represent each convolution weight as a linear combination of Fourier-Bessel bases, and learn the coefficients corresponding to those bases using the MNIST data. Compare the performance with DCF layer that learn the coefficients and filter atoms from scratch.

2. Make a plot: Visualize Fourier-Bessel bases and filter atoms.

Task 3: MNIST Digits Reconstruction Experiment

1. Pretrain an autoencoder [5] on MNIST dataset. The goal of this autoencoder is to reconstruct a clean digit from a noisy digit (a clean digit + Gaussian noise).

2. Replace the convolutional filter in the autoencoder with a DCF layer or Fourier-Bessel bases and conduct experiments.

3. Make a plot: PSNR [6] vs number of components (filter atoms or Fourier-Bessel bases). Specifically, you need to measure the PSNR between the reconstructed clean image and the ground-truth clean image on a held-out dataset.

Discussion

- How many components (filter atoms or Fourier-Bessel bases) are needed to retain performance?

- What is the trade-off between number of parameters and performance (accuracy, or PSNR)?

Part 2: Domain Adaptation with MNIST and SVHN

Goal: Understand **domain shift as subspace shift**, and how shared coefficients enforce invariance.

Task 1: Baseline

1. Train an AlexNet on MNIST, test on SVHN (no adaptation).
2. Report the test accuracy.
3. Note: MNSIT digits have 1 channel and SVHN digits have 3 channels. You can average 3 channels of SVHN digits to make 1-channel images.

Task 2: DCF-Based Adaptation

1. Create a modified AlextNet, where convolutional filters are replaced by DCF layers, i.e., $W = \sum_{k=1}^{K} a_k \psi_k$. In this model, we have Domain-specific atoms (ψ_k) and Shared coefficients (a_k).
2. Train the modified AlexNet directly on SVHN and report test accuracy (upper bound).
3. Train the modified AlextNet first on MNIST, learning both domain-specific atoms and shared coefficients. Then adapt the domain-specific atoms to SVHN and report the test accuracy on SVHN.
4. Make a plot:
    - Vary number of atoms K, show test accuracy on SVHN vs K
    - Visualization of learned domain-specific atoms (MNIST vs SVHN)

Discussion

- Why does sharing coefficients promote semantic alignment?
- What happens if coefficients are domain-specific, while atoms are shared instead? Please conduct the experiment and provide the results in a table.

Part 3: Open-Ended Exploration (Choose One)

You need to **formulate your own hypothesis** and validate it experimentally. Use the following topics as a guideline. Discuss your design and findings.

To demonstrate your design experimentally, you may select your own setup, including but not limited to **datasets** such as MNIST, SVHN, or CIFAR, and **tasks** such as classification or reconstruction. You should clearly report your design choices, proposed method, experimental setup, and comparisons with baselines, and present results.

Option A: Continual Learning with Component Decomposition

Goal: Consider a sequence of tasks in which a single model is expected to incrementally learn new tasks without forgetting previously acquired knowledge. For example, on the MNIST dataset, a model may first be trained on digits 0–1 and then adapted sequentially to digits 2–3, 4–5, and so on. A naïve approach would train and store a separate model for each subtask, but this incurs a substantial memory overhead. Design a continual learning method based on the component decomposition framework discussed above.

You should present results including accuracy (or PSNR) versus number of tasks, memory footprint, and forgetting curves.

Discussion

- Why does storing subspaces prevent catastrophic forgetting?
- How does this technique extend to your own research interests?

Option B: Beyond PCA – New Parameter Space Decompositions

Goal: In this project, we mainly discuss using PCA to achieve filter decomposition. However, you can choose other methods to decompose filters. For example, given convolutional filters, you can just apply random projection, or other matrix factorization methods, or sparse coding techniques such as K-SVD to decompose convolutional filters.

You should compare the results of your design with DCF, such as memory footprint, orthogonality of bases, and accuracy (or PSNR) on the task.

Deliverables

1. Written Report

- Your understanding of Part 1 and Part 2, including:
    - Clear problem formulation
    - Mathematical description of DCF
    - Experimental setup
    - Plots with interpretation
    - (Optional) Discussion connecting results to theory
- In Part 3, you need to discuss:
    - Motivation for the problem
    - Problem formulation
    - Design choice and logic
    - Experimental settings and results
    - Conclusion

2. Short Presentation (near the end of the semester, including both project 1 & 2, open-ended part)

- 10–12 slides summarizing insights
- Present in class about your work

Reference

[1] Qiu, Q., Cheng, X., & Sapiro, G. (2018). DCFNet: Deep neural network with decomposed convolutional filters. ICML.

[2] Wang, Z., Cheng, X., Sapiro, G., & Qiu, Q. (2020). A dictionary approach to domain-invariant learning in deep networks. NeurIPS.

[3] Miao, Z., Wang, Z., Chen, W., & Qiu, Q. (2021). Continual learning with filter atom swapping. ICLR.

[4] Wang, Z., Hwang, S., Miao, Z., & Qiu, Q. (2021). Image generation using continuous filter atoms. NeurIPS.

[5] Materials on autoencoders: http://ufldl.stanford.edu/tutorial/unsupervised/Autoencoders/, https://en.wikipedia.org/wiki/Autoencoder

[6] https://en.wikipedia.org/wiki/Peak_signal-to-noise_ratio